

MongoDB Learning Documentation

Date: November 3, 2025

Table of Contents

1. [Database Operations](#)
 2. [Collection Operations](#)
 3. [CRUD Operations](#)
 4. [Query Operations](#)
 5. [Update Operations](#)
 6. [Delete Operations](#)
 7. [Sorting and Limiting](#)
 8. [Key Takeaways](#)
-

Database Operations

Viewing Databases

```
javascript  
  
show dbs  
// Lists all databases with their sizes
```

Switching/Creating Database

```
javascript  
  
use product  
// Switches to 'product' database (creates if doesn't exist)
```

Current Database

```
javascript  
  
db  
// Shows current database name
```

Dropping Database

```
javascript
```

```
db.dropDatabase()
```

```
// Deletes the current database
```

```
// Returns: { ok: 1, dropped: 'database_name' }
```

Note: A database only appears in `show dbs` after creating at least one collection in it.

Collection Operations

Creating Collections

```
javascript
```

```
db.createCollection('Books')
```

```
// Creates a new collection named 'Books'
```

```
// Returns: { ok: 1 }
```

Viewing Collections

```
javascript
```

```
show collections
```

```
// Lists all collections in current database
```

Dropping Collections

```
javascript
```

```
db.collectionName.drop()
```

```
// Deletes the specified collection
```

```
// Returns: true (if successful)
```

CRUD Operations

Insert Operations

Insert Single Document

```
javascript
```

```
db.Books.insertOne({
  name: "Cpp",
  author: "Swet"
})
// Returns acknowledgment with insertedId
```

Insert with Multiple Fields

```
javascript

db.Books.insertOne({
  name: "C",
  author: "rahul",
  price: 500
})
```

Insert with Array Field

```
javascript

db.Books.insertOne({
  name: "Python",
  author: ["rahul", "shyam"],
  price: 1000
})
// Arrays can be used for fields with multiple values
```

Count Documents

```
javascript

db.Books.countDocuments()
// Returns the total number of documents in collection
```

Query Operations

Find All Documents

```
javascript

db.Books.find()
// Returns all documents in the collection
```

Pretty Print

```
javascript
```

```
db.Books.find().pretty()  
// Formats the output for better readability
```

Query with Filters

Equality Operator

```
javascript
```

```
db.Books.find({price: {$eq: 1000}})  
// Finds documents where price equals 1000
```

Greater Than

```
javascript
```

```
db.Books.find({price: {$gt: 0}})  
// Finds documents where price is greater than 0
```

Greater Than or Equal

```
javascript
```

```
db.Books.find({price: {$gte: 0}})  
// Finds documents where price is greater than or equal to 0
```

Common Query Operators

- `$eq` - Equal to
- `$gt` - Greater than
- `$gte` - Greater than or equal to
- `$lt` - Less than
- `$lte` - Less than or equal to
- `$ne` - Not equal to

Update Operations

Update with \$set Operator

```
javascript
```

```
db.Books.updateOne(
  { _id: ObjectId('690884961738fa1b9096a68e') },
  {
    $set: {
      name: "The C programming",
      author: "Raj",
      price: 1000
    }
  }
)
```

Important: Update operations require atomic operators like `$set`, `$unset`, `$inc`, etc.

✗ Wrong:

```
javascript

db.Books.updateOne(
  { _id: ObjectId('...') },
  { name: "New Name" } // This will throw error
)

// Error: Update document requires atomic operators
```

✓ Correct:

```
javascript

db.Books.updateOne(
  { _id: ObjectId('...') },
  { $set: { name: "New Name" } }
)
```

Rename Field

```
javascript

db.Books.updateOne(
  { _id: ObjectId('690884961738fa1b9096a68e') },
  {
    $rename: {
      name: "bookname"
    }
  }
)

// Renames the 'name' field to 'bookname'
```

Update Return Object

```
javascript

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1, // Number of documents matched
  modifiedCount: 1, // Number of documents modified
  upsertedCount: 0
}
```

Delete Operations

Delete Single Document

```
javascript

db.Books.deleteOne({_id: ObjectId('690885501738fa1b9096a68f')})
// Deletes one document matching the filter
```

Deprecated Method

```
javascript

db.Books.remove({_id: ObjectId('...')})
// ⚠️ DEPRECATED: Use deleteOne() or deleteMany() instead
```

Recommended Methods:

- `deleteOne()` - Deletes first matching document
- `deleteMany()` - Deletes all matching documents
- `findOneAndDelete()` - Finds and deletes, returning the deleted document

Sorting and Limiting

Limit Results

```
javascript

db.Books.find().limit(1)
// Returns only 1 document
```

Sort Ascending

```
javascript  
  
db.Books.find().sort({price: 1})  
// Sorts by price in ascending order (1 = ascending)
```

Sort Descending

```
javascript  
  
db.Books.find().sort({price: -1})  
// Sorts by price in descending order (-1 = descending)
```

Sort by Text Field

```
javascript  
  
db.Books.find().sort({name: 1})  
// Sorts alphabetically by name
```

Chaining Methods

```
javascript  
  
db.Books.find().sort({price: -1}).limit(5).pretty()  
// Chain multiple methods for complex queries
```

Key Takeaways

Document Structure

- Documents in MongoDB are flexible (schema-less)
- Each document automatically gets a unique `_id` field (ObjectId)
- Fields can have different data types including arrays and nested objects
- Different documents in same collection can have different fields

Best Practices

1. Always use atomic operators (`$set`, `$unset`, etc.) for updates
2. Use `deleteOne()` or `deleteMany()` instead of deprecated `remove()`
3. Use `.pretty()` for readable output during development

4. Count documents with `countDocuments()` instead of deprecated `count()`

Common Errors Encountered

- 1. **Update without atomic operators** - Always use `$set`, `$inc`, etc.
- 2. **Database not showing in `show dbs`** - Create at least one collection first
- 3. **Wrong syntax in find queries** - Always wrap filter in curly braces: `{field: value}`

MongoDB Shell Commands Summary

Operation	Command
Show databases	<code>show dbs</code>
Use/Create database	<code>use dbname</code>
Current database	<code>db</code>
Show collections	<code>show collections</code>
Create collection	<code>db.createCollection('name')</code>
Insert document	<code>db.collection.insertOne({})</code>
Find all	<code>db.collection.find()</code>
Update	<code>db.collection.updateOne({filter}, {\$set: {}})</code>
Delete	<code>db.collection.deleteOne({filter})</code>
Count	<code>db.collection.countDocuments()</code>
Drop collection	<code>db.collection.drop()</code>
Drop database	<code>db.dropDatabase()</code>

Sample Dataset Used

```
javascript
```

// Document 1

```
{
  _id: ObjectId('690884251738fa1b9096a68d'),
  name: 'Cpp',
  author: 'Swet'
}
```

// Document 2

```
{
  _id: ObjectId('690884961738fa1b9096a68e'),
  author: 'Raj',
  price: 1000,
  bookname: 'The C programming'
}
```

// Document 3 (deleted)

```
{
  _id: ObjectId('690885501738fa1b9096a68f'),
  name: 'Python',
  author: ['rahul', 'shyam'],
  price: 1000
}
```

Next Steps for Learning

- ☐ Learn aggregation pipeline
- ☐ Explore indexing for performance
- ☐ Study relationships (embedded vs referenced)
- ☐ Practice with complex queries using `$and`, `$or`
- ☐ Learn about MongoDB transactions
- ☐ Explore MongoDB Atlas (cloud database)

Documented by: [Your Name]

Date: November 3, 2025

MongoDB Shell Version: mongosh